

Reinforcement Learning for Portfolio Rebalancing

Review 2: Progress Report

Surbhi Agarwal (J002) | Urja Innani (J018) | Taha Lokhandwala (J024)

Abstract

This report describes the progress made so far on our project, which studies whether a reinforcement learning agent can rebalance a portfolio of Indian exchange traded funds better than fixed rules once realistic trading costs are included. At this stage we have built the data pipeline, an Indian transaction cost model, a portfolio simulator and a set of benchmark strategies. The simulator passes all five sanity tests and a first trial run of the agent is complete. The main experiments and final evaluation follow in the next phase.

1. Introduction

An investor who holds several funds has to decide how often to bring the portfolio back to its planned mix. Prices move at different speeds, so over time some holdings grow larger than intended and others shrink. Restoring the original weights is called rebalancing. Rebalancing too often wastes money on brokerage, taxes and other charges, while rebalancing too rarely lets the portfolio drift into a riskier position than the investor wanted.

In practice most investors follow a simple calendar rule, for example rebalancing every quarter, regardless of market conditions. Such rules are easy to follow but ignore information that is available at the time of the decision. Our project asks whether a reinforcement learning (RL) agent can learn *when* and *how much* to rebalance, and whether this leads to better risk-adjusted returns than fixed rules after real transaction costs are taken into account.

2. Background

Reinforcement learning is a branch of machine learning in which an agent learns by interacting with an environment. At each step the agent observes the current state, takes an action and receives a reward. Over many episodes it adjusts its behaviour to collect as much reward as possible. Portfolio management fits this setting naturally: the state is recent market information and the current holdings, the action is the trade to make, and the reward is the resulting profit or loss.

Earlier work has applied RL directly to portfolio selection, for example the EIIE framework of Jiang et al. (2017). Classical approaches instead compute weights from estimates of risk and return, such as mean-variance optimisation, risk parity (Maillard et al., 2010) and hierarchical risk parity (Lopez de Prado, 2016). These methods say where the portfolio should be but ignore the cost of getting there; our work combines the two ideas.

3. Objectives

- Build a realistic simulator of a weekly-rebalanced portfolio of Indian ETFs, including Indian trading costs.
- Implement a set of standard rebalancing rules and classical optimisers to serve as benchmarks.
- Train an RL agent that decides the timing and size of rebalancing trades.
- Evaluate all strategies on the same data and costs, strictly out of sample.

4. Methodology

4.1 Data

We use weekly closing prices of seven ETFs listed on the National Stock Exchange, together with a cash position that earns a risk-free rate of 6.5% per year. The universe covers large and mid cap Indian equity, banking, public sector banks, infrastructure, gold and US technology stocks, which gives a reasonable spread of risk. Prices are downloaded from Yahoo Finance and saved as a frozen snapshot with a content hash, so every experiment uses exactly the same data.

The data is split in time order into three periods. The training period (up to December 2019) is used for learning, the validation period (2020 to 2021) is used to choose the best version of each agent, and the test period (January 2022 onwards) is kept locked until the final evaluation. A gap of four weeks is left between periods so that information from one period cannot leak into the next.

Ticker	Fund	Asset class
NIFTYBEES	Nippon India Nifty 50 BeES	Large cap equity
JUNIORBEES	Nippon India Nifty Next 50 BeES	Large / mid cap equity
BANKBEES	Nippon India Nifty Bank BeES	Banking
PSUBNKBEEES	Nippon India Nifty PSU Bank BeES	Public sector banks
INFRABEES	Nippon India Nifty Infra BeES	Infrastructure
GOLDBEES	Nippon India Gold BeES	Gold
MON100	Motilal Oswal Nasdaq 100 ETF	International equity

Table 1: Investment universe

4.2 Transaction cost model

Every trade is charged according to current Indian rules for ETF delivery trades through a discount broker. The same model is applied to the agent and to every benchmark, so the comparison is fair.

Charge	Rate (% of value traded)	Applied on
Securities Transaction Tax (STT)	0.001%	Sell side
Stamp duty	0.015%	Buy side
NSE transaction charge	0.00307%	Both sides
SEBI turnover fee	0.0001%	Both sides
GST	18% of exchange and SEBI charges	Both sides
Slippage allowance	0.05% (5 bps)	Both sides

Table 2: Transaction cost model

4.3 Environment and agent design

State. Each week the agent sees the last 20 weeks of standardised returns for every ETF, recent volatility and correlations, its current weights, the current drawdown and the target weights proposed by the optimiser. To avoid look-ahead bias, the agent only uses data available up to the previous week.

Action. We use a hybrid design. A classical optimiser (risk parity) proposes a target portfolio each week, and the agent chooses a single number a between 0 and 1, the fraction of the gap to that target to close. A value of 0 means no trade and 1 means a full rebalance. This keeps the action small and easy to interpret, and makes it harder for the agent to exploit flaws in the simulator. No single ETF may exceed 40% of the portfolio. Costs are charged on the trade before prices move to the next week, so the agent cannot trade at prices it has already seen.

Reward. The weekly reward is the log return of the portfolio after costs, minus a penalty on losses: $reward = \log(1 + r) - \lambda \cdot \min(r, 0)^2$, with $\lambda = 5$. Squaring the loss means that one large loss is penalised much more than several small ones, while gains are never penalised. Alternative penalties (variance and drawdown) are also implemented for later comparison.

4.4 Training protocol

Agents are trained with the PPO algorithm from Stable-Baselines3 on episodes of 104 weeks with random starting points inside the training period. Each configuration is trained with several random seeds. During training the agent is evaluated on the validation period every 5,000 steps and the best checkpoint is kept. Results are reported as the median across seeds, and a deflated Sharpe ratio (Bailey and Lopez de Prado, 2014) is used to adjust for the number of configurations tried.

5. System Implementation

The project is implemented as a web application. The backend is written in Python using FastAPI, with NumPy and pandas for data handling, Gymnasium for the environment and Stable-Baselines3 with PyTorch for training. The React frontend has a page for each stage of the work, and every training run is saved with its settings so that experiments can be compared later.

6. Work Completed and Results So Far

The data pipeline, cost model, simulator, nine benchmark strategies, five sanity tests, the RL training pipeline and the web interface are complete. The results obtained so far are summarised below.

6.1 Benchmark results

Strategy	Sharpe	CAGR	Max drawdown	Turnover / yr
Risk parity, monthly	0.73	19.1%	19.1%	91%
HRP, monthly	0.64	16.6%	17.8%	101%
Market index (NIFTYBEES)	0.64	20.8%	33.5%	51%
Equal-weight, quarterly	0.63	19.2%	26.1%	64%
Minimum-variance, monthly	0.62	16.3%	17.2%	148%
Equal-weight, monthly	0.60	18.5%	27.1%	75%
Equal-weight, 5% band	0.59	18.2%	26.7%	58%
Buy and hold	0.58	17.4%	26.1%	51%
Mean-variance, monthly	0.29	10.9%	22.2%	321%

Table 3: Benchmark strategies, validation period 2020 to 2021, net of costs

Quarterly equal-weight rebalancing is the rule most often used in practice, so we treat its Sharpe ratio of 0.63 as the benchmark the agent has to beat. The market index earned the highest return but lost about a third of its value in the COVID crash, while minimum-variance and HRP limited the fall to under 18% at the cost of lower returns. Mean-variance did worst, mainly because it traded heavily. This trade-off between return, risk and trading is exactly what the agent is meant to manage.

6.2 Sanity tests

Test	What it checks	Result
1. Buy and hold	Simulator value matches a hand calculation with zero costs	Pass
2. Forced rotation	Costs charged equal turnover times the cost rate	Pass
3. Equal weight	An equal-weight action reproduces the equal-weight portfolio	Pass
4. Shuffled returns	Agent trained on shuffled data finds no timing edge	Pass
5. Look-ahead	Agent given next week's returns performs far better	Pass

Table 4: Sanity tests on the simulator and learning pipeline

7. Preliminary Observations

A first trial run of the agent (PPO, two random seeds, 20,000 training steps each) completed successfully. On the validation period it followed the benchmarks closely and had a smaller drawdown during the 2020 crash. The two seeds gave very similar results. This was a small, untuned run, so we do not draw conclusions from it yet; it mainly confirms that the pipeline works end to end.

8. Challenges Faced

- **Limited common history.** Reliable data for all seven ETFs is only available from May 2018, which leaves a short training period. We are considering moving the split dates or changing the universe.
- **Data quality.** Some price series contained unadjusted unit splits, which had to be detected and repaired.
- **Overfitting.** Financial data is noisy and RL results vary between runs, so we rely on several seeds, a separate validation period and a deflated Sharpe ratio.

9. Remaining Work

- **Full training:** more seeds (5 to 10) and longer training runs.
- **Algorithm study:** PPO, A2C, SAC, TD3 and DQN, with MLP, EIIE and attention networks.
- **Reward study:** downside, variance and drawdown penalties at different values of λ .
- **Robustness:** higher transaction costs, different market regimes and walk-forward testing.
- **Final evaluation and report:** one-time evaluation on the locked test period, analysis and final report.

References

- Bailey, D. H. and Lopez de Prado, M. (2014). The deflated Sharpe ratio. *Journal of Portfolio Management*, 40(5).
- Jiang, Z., Xu, D. and Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. arXiv:1706.10059.
- Lopez de Prado, M. (2016). Building diversified portfolios that outperform out of sample. *Journal of Portfolio Management*, 42(4).
- Maillard, S., Roncalli, T. and Teiletche, J. (2010). The properties of equally weighted risk contribution portfolios. *Journal of Portfolio Management*, 36(4).
- Schulman, J. et al. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.